



Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Unidade Curricular de Interface Pessoa-Máquina

Ano Letivo de 2024/2025

Trabalho prático - Fase 2 Grupo 10

Marco Soares Gonçalves (a104614)
Salvador Duarte Magalhães Barreto (a104520)
André Filipe Soares Pereira (a104275)
Leonardo Gomes Alves (a104093)

4 de maio de 2025

IPM

Data de Receção	
Responsável	
Avaliação	
Observações	

Trabalho prático - Fase 2

Grupo 10

Marco Soares Gonçalves (a104614)
Salvador Duarte Magalhães Barreto (a104520)
André Filipe Soares Pereira (a104275)
Leonardo Gomes Alves (a104093)

4 de maio de 2025

Índice

1	Introdução e Objetivos	2
2	Implementação	3
2.1	Estrutura do projeto e bibliotecas utilizadas	3
2.2	Alterações sobre o protótipo	5
2.3	Reflexão sobre a aplicação	6
2.3.1	Pontos Fortes	6
2.3.2	Pontos Fracos	7
3	Manual de utilização	9
3.1	Cenário 1	9
3.1.1	Login como Diretor de Curso	9
3.1.2	Consulta do resultado da geração dos turnos	10
3.1.3	Análise das UCs com ocupação desequilibrada	10
3.1.4	Identificação de alunos não alocados	11
3.1.5	Alocação manual de alunos	12
3.1.6	Identificação de alunos com colisões	13
3.1.7	Troca de turnos	13
3.1.8	Publicação dos horários dos alunos	14
3.2	Cenário 2	14
3.2.1	Receção do pedido do docente	14
3.2.2	Consulta do horário da UC e das salas disponíveis	15
3.2.3	Alteração da sala do turno	16
3.2.4	Confirmação da alteração ao docente	16
3.3	Cenário 4	17
3.3.1	Login como Aluno	17
3.3.2	Consulta do horário de aulas	18
3.3.3	Envio do pedido de mudança de turno	18
3.3.4	Confirmação do pedido pelo Diretor de Curso	19
3.4	Funcionalidades Adicionais	20
3.4.1	Remoção Manual de Aluno de um Turno	20
4	Conclusão	22



Marco Gonçalves
(a104614)



Leonardo Alves
(a104093)



Salvador Barreto
(a104520)



André Pereira
(a104275)

Link Figma: <https://www.figma.com/design/cLMLElSA17bEqGWQDesfu/Untitled?node-id=0-1&t=2U1Q8PBTYqBv1FVd-1>

Link Github: https://github.com/UMinho-ENGINF-IPM/trabalho-pr-tico-gp25_10

1 Introdução e Objetivos

Foi-nos proposto desenvolver um sistema que auxilie o diretor de curso na gestão de distribuição de alunos pelos diversos turnos existentes.

Para além deste objetivo principal, o grupo decidiu dar ênfase à criação de uma interface intuitiva e acessível, que permita ao diretor de curso obter uma visão geral do estado atual do sistema, bem como a facilidade de acessibilidade às diferentes funcionalidades disponíveis.

Este desenvolvimento teve como base os princípios de usabilidade estudados ao longo da unidade curricular e foi baseada no protótipo desenvolvido no **Figma** e no feedback recebido após o checkpoint.

Com este relatório pretendemos documentar o trabalho realizado para desenvolver o sistema, incluindo as nossas decisões de design e implementação, pontos fortes e fracos do ponto de vista de um utilizador, acompanhado por um manual de utilização do sistema.

2 Implementação

No seguinte capítulo iremos abordar a implementação da nossa aplicação, desde as tecnologias utilizadas, à explicação das funcionalidades desta.

2.1 Estrutura do projeto e bibliotecas utilizadas

No que diz respeito à implementação da aplicação, utilizámos as seguintes tecnologias.

- JSON Server - De forma a conseguir guardar os dados da aplicação, recorreremos ao ficheiro JSON fornecido pela equipa docente e, efetuamos algumas alterações de forma a facilitar a implementação;
- Vue.js - Utilizámos a framework Vue.js para o desenvolvimento da interface do utilizador, tirando partido da sua abordagem reativa e da integração com HTML e CSS;
- Python - Ainda que não esteja presente no repositório, utilizámos pequenos script's Python de forma a ser possível normalizar o dataset, isto é, fazer a associação automática dos alunos a turnos.

Passando agora para uma explicação mais detalhada do desenvolvimento da aplicação em Vue.js, utilizámos as bibliotecas **pinia** e **pinia-plugin-persistedstate** para ser possível guardar estados da aplicação. Relativamente ao segundo, apenas foi utilizado para guardar os dados de autenticação do utilizador, ou seja, caso o utilizador esteja autenticado e feche o navegador, assim que o abrir, estará automaticamente com sessão iniciada no navegador. O **pinia** foi ainda utilizado para o pré-preenchimento de parâmetros de algumas páginas.

Utilizámos ainda a biblioteca **vue-router**, de forma a definirmos as rotas do nosso programa.

No que diz respeito à divisão de componentes e organização na estrutura do nosso projeto, seguimos uma arquitetura modular, com pastas específicas para cada responsabilidade. É ainda importante referir que, o desenvolvimento do código foi maioritariamente feito utilizando a abordagem **Composition API**.

De seguida, explicamos a estrutura das pastas do projeto:

- **images** - Pasta composta pelas imagens utilizadas pelo *website*;
- **public** - Pasta composta por ficheiros estáticos acessíveis diretamente pelo navegador. No nosso caso, apenas é composto pelo *favicon*;
- **src** - Pasta principal do projeto, onde se encontra o código relativo ao Vue.js.
 - **assets** - Pasta composta por recursos estáticos utilizados na aplicação, como por exemplo, ficheiros CSS responsáveis por definirem os estilos das páginas;
 - **components** - Pasta composta por recursos reutilizáveis na aplicação. No nosso caso, temos a **SideBar.vue** e **TerminarSessao.vue** que são precisamente a barra lateral, utilizada em todo o nosso programa, e a funcionalidade de terminar sessão e voltar à página inicial, respectivamente. Por fim, existe ainda o **Horario.vue** que é responsável por desenhar o horário de um certo aluno da aplicação.
 - **models** - Pasta composta por estruturas de dados utilizadas essencialmente para o controlo de permissões por parte de certo tipo de utilizadores. No nosso caso, temos o **Aluno.js** e **Diretor.js**.
 - **router** - Pasta composta por um único ficheiro responsável por determinar as rotas do programa, utilizando o **vue-router**.
 - **stores** - Pasta composta pelas diferentes *stores* utilizadas pelo programa. Em cada um dos ficheiros desta pasta, é utilizada a função `defineStore` do *pinia* para criar as *stores* e, no caso do ficheiro **auth.js**, é utilizado o **pinia-plugin-persistedstate** para persistir o estado.
 - **views** - Pasta composta pelas páginas e lógica que liga o back-end ao front-end. É nesta que são desenhadas as páginas e implementada a lógica de interação com os dados provenientes do JSON server.

Quanto ao acesso e execução da aplicação, é necessário executar o nosso json-server em primeiro lugar, que pode ser acessado através da porta 3000.

```
json-server -w trabalhodob.json
```

Por fim, dentro da pasta `gestaoHorarios` temos que executar o comando **npm run dev** para correr a aplicação na porta 5173.

2.2 Alterações sobre o protótipo

Apesar de seguirmos o protótipo durante a implementação da aplicação, sentimos a necessidade de realizar algumas alterações:

- Simplificámos o processo de seleção de turnos nos menus de alocação manual, troca de turnos e pedido de troca por parte do aluno. Trocamos assim, os menus de seleção de UC e turno por *drop-downs* na mesma página. Esta alteração teve como objetivo adotar um design mais minimalista, reduzindo o número de passos necessários para executar uma ação simples.
- Nos mesmos menus referidos anteriormente, adicionámos a visualização do horário do aluno. Esta funcionalidade permite ao utilizador ter uma perceção mais clara do impacto da seleção de um determinado turno, evitando a criação de colisões, respeitando assim o princípio 6 das heurísticas de Nielsen.
- No menu de visualização de alunos com colisões de horário, passámos a apresentar, para cada aluno, uma caixa com as colisões detetadas. Desta forma, o utilizador pode visualizar os conflitos antes de aceder ao menu de resolução.
- Adicionamos a opção de remover um aluno de um turno, facilitando a gestão de casos em que o aluno não deve estar alocado a nenhum turno ou quando é necessário reorganizar manualmente a distribuição.
- Removemos a funcionalidade de um Diretor de Curso realizar um pedido novo, pois considerarmos que esse tipo de ação cabe apenas aos alunos e professores. O papel do Diretor de Curso será, assim, o de responder e gerir os pedidos submetidos.
- Relativamente aos *pop-ups* que apresentamos no protótipo, substituímos estes por alerts do navegador, por ser mais fácil a sua implementação.
- Quando o Diretor de Curso acede ao menu de detalhes do aluno, aparece agora um botão que permitirá ao utilizador visualizar o horário do aluno em questão.

2.3 Reflexão sobre a aplicação

Nesta secção dividimos a reflexão da nossa aplicação em pontos positivos e pontos negativos.

2.3.1 Pontos Fortes

A aplicação apresenta diversas qualidades, nesta secção destacamos os principais pontos fortes da nossa solução:

- No nosso modelo inicial desenvolvido no **Figma**, existiam vários aspetos que foram melhorados após o feedback no checkpoint. Para tornar mais fácil a troca de turnos, o diretor de curso passou agora a ter uma visão lado a lado do horário com as UC 's que pretende trocar. Deste modo, ele consegue perceber facilmente se a troca será problemática (gerar conflitos/ultrapassar capacidade do turno). Além disso, adicionamos também esta visão na alocação manual de turno, devido a razões semelhantes e ainda trocamos os botões de seleção que, anteriormente, levavam a páginas desnecessárias e inconvenientes para um utilizador por dropdowns, eliminando a necessidade de decorar o horário do aluno. Fora isso, fomos o mais fieis possível ao protótipo **Figma** e consideramos que a interface será facilmente percebida por qualquer utilizador que tenha o mínimo de conhecimento sobre o sistema escolar.
- Garantimos que as informações são apresentadas de forma organizada e visualmente compreensível. Os horários são apresentados em um formato intuitivo, que deixa colisões de turnos óbvias. Nas diferentes listas de alunos, para além de um sistema de paginação, possuímos também uma barra de pesquisa, que permite ao diretor de curso procurar por um utilizador específico através do seu nome e/ou identificador único. Nas listagens de alunos com horários incompletos/ com colisões, estes são apresentados com elementos visuais que deixam claros os turnos pertinentes e que permitem aceder facilmente ao horário do aluno para ter o panorama total do estado do seu horário. Facilitam também a troca/alocação de turnos para os horários com colisões e os horários incompletos respetivamente; finalmente nestas páginas deixamos clara a identificação de alunos que tivessem estatuto associado simplificando o processo de prioridade por parte do Diretor. Nos pedidos do Diretor fizemos algumas alterações à terminologia utilizada e uma melhoria nas páginas de pedidos pendentes e pedidos concluídos ao tornar a cor predominante em cada página óbvia para os utilizadores, uma vez que antes a troca entre as páginas era quase impercetível dada a paleta de cores idêntica(pendentes a vermelho / concluídos a verde).
- No que toca à navegação pelas páginas da aplicação tentamos tornar a mesma consistente e previsível. A qualquer momento o utilizador pode usufruir do menu lateral para voltar à página principal associada a cada uma das opções no mesmo; do mesmo modo a opção associada à página em que se encontra estará sempre demarcada com um azul escuro em contraste com as outras opções que permanecem cinzentas. Ade-

mais, o botão de voltar pode também ser utilizado se o utilizador pretender retornar a uma página que visitou anteriormente. Como mencionado previamente nas páginas de visualização de alunos com horários com conflitos ou incompletos, é disponibilizado um botão que permite simplificar o processo de retificação desses problemas ao, não só redirecionar o utilizador para a página onde pode fazê-lo, como também ao pré-selecionar o aluno relevante. Nas páginas que permitem trocar turnos, atribuí-los manualmente ou removê-los de um horário, a conclusão desses processos leva a um recarregamento automático que revela o horário atualizado garantindo a coerência e confirmação das ações realizadas.

- Todas as ações que o utilizador tome são acompanhadas por mensagens de feedback claras e imediatas, que indicam se a operação foi concluída com sucesso, se a operação falhou, bem como a explicação da causa da falha. Para além disso foram adicionados warnings que avisam o utilizador quando uma ação que este pretende tomar possa gerar situações problemáticas, ou os campos a preencher não estão completos.
- Todos os dropdowns identificam os campos a preencher caso estejam desativados, deixando claro ao utilizador o porquê de não poder realizar as ações numa ordem arbitrária, por exemplo, para escolher um turno é obrigatória a seleção prévia da UC associada.
- A terminologia foi consistente e acessível dado o contexto do trabalho e deixa poucas dúvidas quanto àquilo que possam referir ou pretender informar, exceto nos horários onde apenas temos a abreviação de cada disciplina o que poderia causar confusão aos alunos ou mesmo o Diretor de Curso.

2.3.2 Pontos Fracos

Apesar dos aspetos positivos, identificámos também algumas limitações que enfrentámos ao longo do desenvolvimento. Nesta secção, refletimos sobre os pontos fracos da nossa aplicação:

- Apesar da melhoria dos dropdowns, reconhecemos que ainda existem algumas tarefas que poderiam ser melhoradas de modo a oferecer uma melhor experiência ao utilizador, tal como por exemplo, uma lista de avisos na página inicial do diretor de curso, para que assim que este inicie a aplicação saiba os problemas que necessita de resolver.
- Ao longo do trabalho o nosso foco nunca foi em eficiência, algo que é evidente para qualquer utilizador que use o nosso programa. O horário demora alguns instantes a ser apresentado e o carregamento de algumas páginas também não é espontâneo, sendo que apresentam uma mensagem durante um instante antes de ser atualizada com os elementos reais, mensagem esta que deve aparecer apenas quando não há elementos a apresentar, por exemplo, na lista de horários incompletos se não existir qualquer horário incompleto.
- Outra grande falha do nosso sistema é a falta de utilização de media queries, o que

compromete a consistência do website entre diferentes dispositivos. Dependendo das proporções do ecrã utilizado o tamanho do conteúdo apresentado varia drasticamente. Esta limitação deve-se a um mau planeamento do design do sistema, visto que decidimos deixar esta “standardização” para o fim e acabamos por não a conseguir implementar. Vale notar que um ajuste do zoom do browser utilizado consegue, normalmente, ajustar a disposição da interface para àquela pretendida.

- O layout é demasiado simples ao ponto que o espaço do ecrã não é utilizado de forma eficiente sendo que a maioria dos elementos demonstram ser desnecessariamente grandes ou o espaço encontra-se mal aproveitado e essa tendência é inerente ao estilo que tomamos para a aplicação.
- Não implementamos medidas de acessibilidade consideráveis, apenas mantemos um design básico com cores/ tamanhos de fonte e componentes do UI acessíveis ; no entanto é importante mencionar que tivemos atenção aos níveis de contraste.
- Outro aspeto que nos falta seria uma opção de exportação de horário, que poderia ser muito útil para os alunos e também um sistema robusto de avisos automáticos feito pelo sistema (por exemplo quando o diretor corresponde ao pedido de troca de turno de um aluno, este precisa de responder ao pedido recebido. O sistema poderia fazer isso).

3 Manual de utilização

Para desenvolver o manual de utilização da aplicação que desenvolvemos, apoiamos-nos nos cenários dados no enunciado, e mostramos como a nossa aplicação satisfaz os mesmos

3.1 Cenário 1

3.1.1 Login como Diretor de Curso

O Diretor de Curso inicia sessão na aplicação utilizando o seu email e palavra-passe.

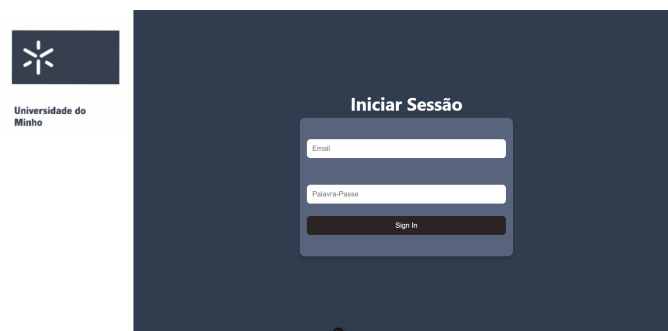


Figura 3.1: Ecrã de login

Caso o utilizador erre as suas credenciais, o seguinte ecrã é apresentado:

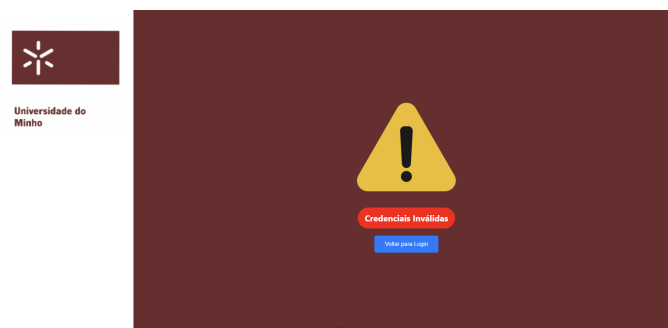


Figura 3.2: Credenciais inválidas

3.1.2 Consulta do resultado da geração dos turnos

Após o login, o diretor é direcionado para o painel principal, onde do lado esquerdo tem acesso ao menu lateral, cuja uma das opções é "Consultar horário de UC".



Figura 3.3: Página inicial

3.1.3 Análise das UCs com ocupação desequilibrada

Clicando em "Consultar horário de UC", o diretor de curso é redirecionado para uma página onde estão listadas todas as unidades curriculares do curso.



Figura 3.4: Consultar horário de UC

Após seleccionar qualquer uma das UCs, são apresentados ao utilizador os seus turnos, informa-

ção relativa aos seus horários, capacidade e sala, como também opções de "Consultar Alunos" e "Editar Turno".



Figura 3.5: Consultar turnos de UC

3.1.4 Identificação de alunos não alocados

Clicando em "Gestão de Alunos" o diretor é redirecionado para o menu onde pode ver colisões, horários incompletos, os alunos inscritos no curso, alocar turnos manualmente, trocar turnos e remover um aluno de um turno.



Figura 3.6: Gestão de Alunos

Após selecionar "Alunos com horário incompleto", o diretor de curso pode observar uma lista de alunos que ficaram com turnos por alocar e os repetitivos turnos em falta. Os alunos marcados por um "*" são alunos com estatuto especial.



Figura 3.7: Alunos com horário incompleto

3.1.5 Alocação manual de alunos

O diretor pode então optar por clicar em "Alocar Turno", um atalho, que o redirecionará para a página de alocação manual de turnos, pre-preenchida com o aluno respetivo.



Figura 3.8: Alocação manual de turnos

Aqui pode seleccionar a UC e o respetivo turno a que pretende alocar o aluno, tendo sempre disponível, do seu lado direito, o horário do aluno.

3.1.6 Identificação de alunos com colisões

Voltando a "Gestão de Alunos", selecionando "Alunos com colisões", o diretor de curso pode observar uma lista de alunos que ficaram com colisões no seu horário e os repetidas colisões. Os alunos marcados por um "*" são alunos com estatuto especial.



Figura 3.9: Alunos com colisões no horário

3.1.7 Troca de turnos

O diretor pode então optar por clicar em "Alocar Turno", um atalho, que o redirecionará para a página de troca de turnos, pre-preenchida com o aluno respetivo.

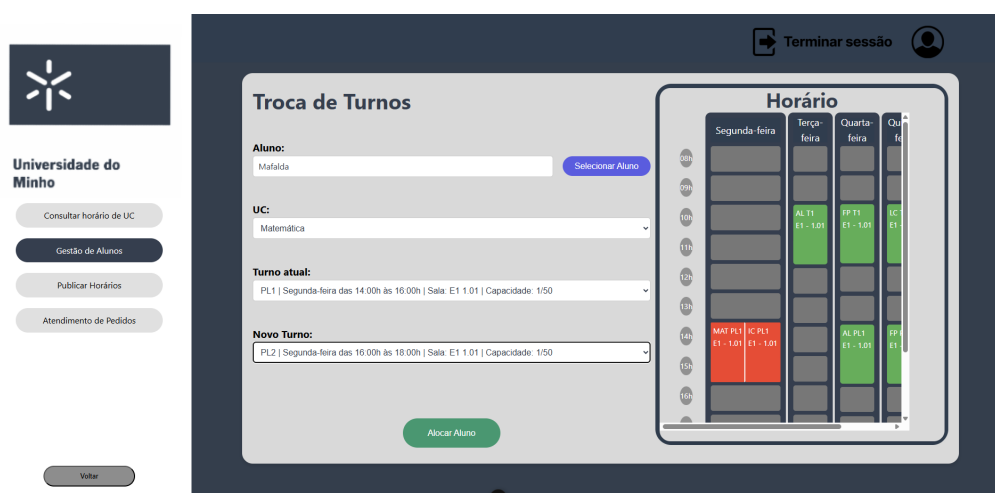


Figura 3.10: Troca de turnos

Aqui pode seleccionar a UC, o turno atual e o turno a que pretende transferir o aluno, tendo sempre disponível, mais uma vez, do seu lado direito, o horário do aluno.

3.1.8 Publicação dos horários dos alunos

Após concluir a alocação, o diretor acede a "Publicar Horários" no menu lateral para ser redirecionado para a página respetiva. Após seleccionar "sim", atualiza os horários e torna-os visíveis para os alunos.

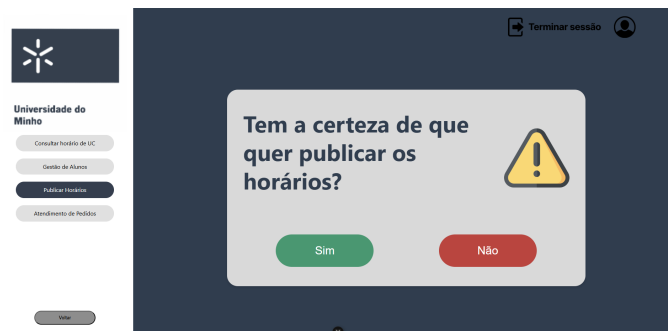


Figura 3.11: Opção de publicação dos horários dos alunos

3.2 Cenário 2

3.2.1 Receção do pedido do docente

O Diretor de Curso, já autenticado na aplicação, acede à secção de pedidos através da opção "Atendimento de Pedidos" no menu lateral. Verifica então que recebeu um pedido de um docente para alterar a sala de um turno de uma UC.

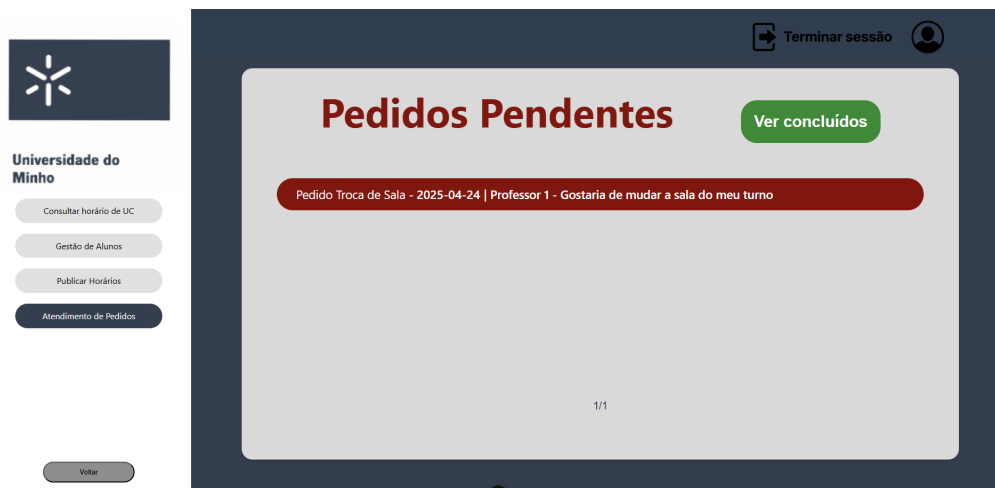


Figura 3.12: Visualização de pedidos pendentes por parte de docentes

Após seleccionar o pedido, pode então ver os seus detalhes.



Figura 3.13: Pedido de troca de Sala

3.2.2 Consulta do horário da UC e das salas disponíveis

O Diretor de Curso acede ao horário da UC e identifica o turno abordado no pedido.



Figura 3.14: Horário de UC

3.2.3 Alteração da sala do turno

Seleciona então a opção "Editar Turno" do turno pretendido. Pode agora então editar o dia da semana, horas e sala do turno.

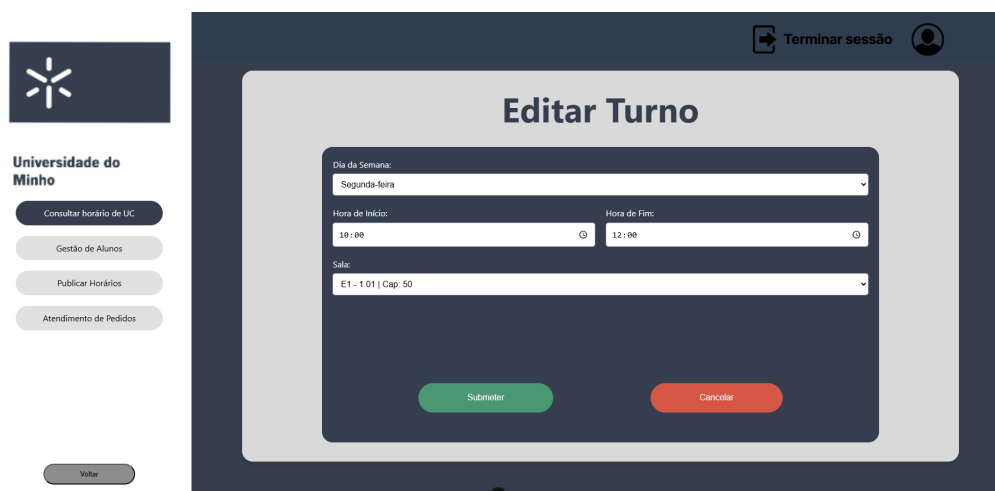


Figura 3.15: Editar turno

3.2.4 Confirmação da alteração ao docente

Após efetuar a alteração e submeter, pode então voltar ao pedido e selecionar a opção "Responder" e preencher o campo "mensagem" com o resultado da operação.



Figura 3.16: Resposta enviada ao docente

Após carregar no botão "Enviar", o pedido vai agora aparecer nos pedidos concluídos, acessíveis através da opção "Ver concluídos" do menu "Atendimento de Pedidos".

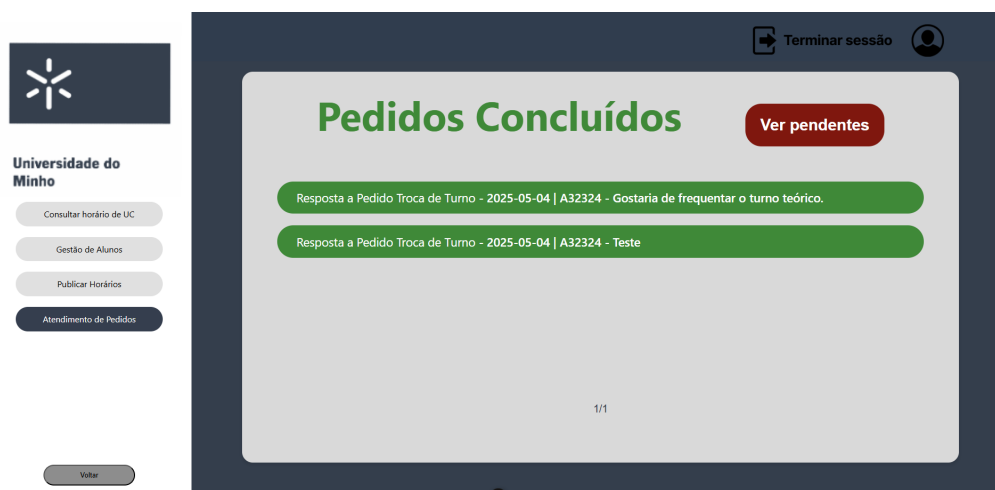


Figura 3.17: Pedidos concluídos

3.3 Cenário 4

3.3.1 Login como Aluno

Um aluno, acede à aplicação utilizando as suas credenciais. Após realizar o login, a aplicação exibe o seu perfil.



Figura 3.18: Página inicial de aluno

3.3.2 Consulta do horário de aulas

O aluno verifica seu horário, através da opção "Consultar Horário" do menu lateral.



Figura 3.19: Consulta do horário atual do aluno

3.3.3 Envio do pedido de mudança de turno

Depois de decidir que deseja alterar um dos seus turnos, acede à opção "Pedidos" do menu lateral, onde são listados os pedidos que ainda não obtiveram resposta.

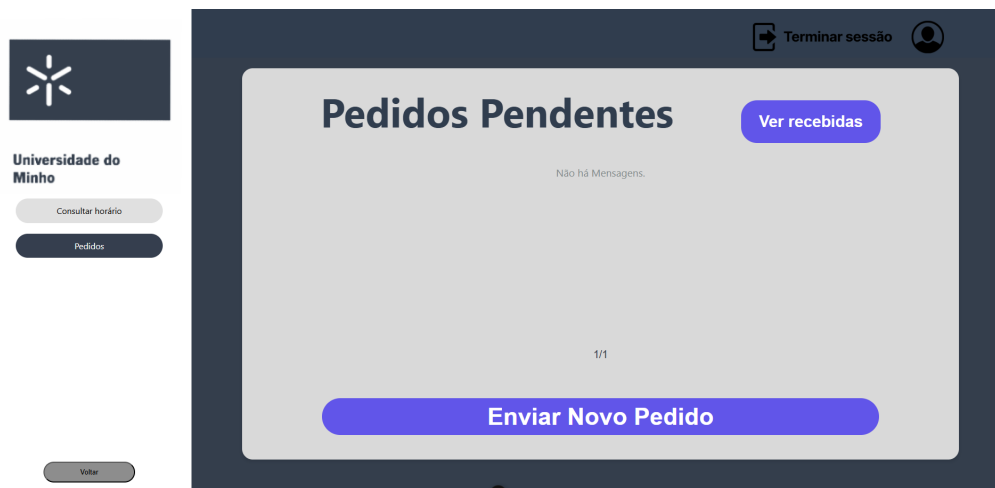


Figura 3.20: Pedidos Pendentes

Carrega então em "Enviar novo pedido", seleciona a UC e o turno para o qual deseja se transferir e pode justificar a troca no campo de mensagem.



Figura 3.21: Novo pedido

3.3.4 Confirmação do pedido pelo Diretor de Curso

Semelhante à visão do diretor de curso, após enviar o pedido este ficará no menu de pedidos pendentes. Após a resposta por parte do diretor de curso, o aluno pode então clicar em "Ver recebidas" para ver os seus pedidos já respondidos.

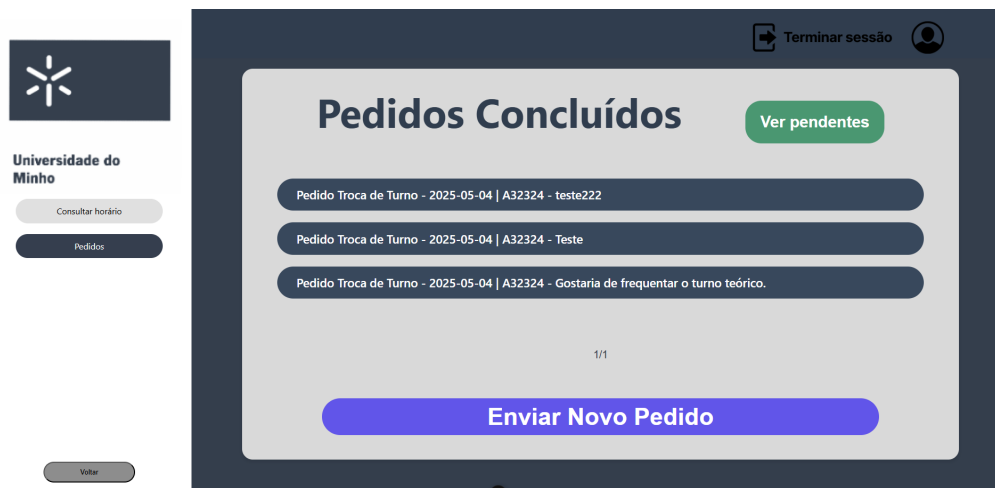


Figura 3.22: Pedidos Concluídos



Figura 3.23: Pedido respondido

3.4 Funcionalidades Adicionais

3.4.1 Remoção Manual de Aluno de um Turno

Esta funcionalidade permite ao Diretor de Curso remover um aluno de um turno específico, na "Gestão de Alunos", selecionando "Remover alocação de aluno".



Figura 3.24: Remoção de aluno num turno

4 Conclusão

Ter realizado este trabalho foi uma experiência enriquecedora para todos nós, uma vez que, a nossa equipa nunca tinha tido contacto com programação de front-end e, por isso, saímos com a consciência que realizámos um bom trabalho relativamente ao que nos foi pedido.

Uma das principais dificuldades da equipa foi a implementação dos estilos CSS, de forma a que estes respeitassem, minimamente, a interface desenvolvida no Figma. Alinhar elementos, garantido sempre a integridade e uniformidade da interface exigiu tentativa e erro, uma vez que nunca tínhamos tido experiência com linguagens como HTML e CSS.

No fundo, as maiores dificuldades foram relativas ao front-end, uma vez que, a utilização de JSON server, por estar mais ligada ao back-end, corresponde a uma área com a qual já estávamos mais familiarizados e onde nos sentimos mais confortáveis, como por exemplo, na implementação da lógica para verificar se, após uma alteração de um certo turno, iria ocorrer um conflito entre turnos.

No entanto, o que mais gostámos de trabalhar com o Vue.js foi a sua simplicidade e estrutura clara e, de certa forma, a linguagem de Javascript também é bastante acessível, o que facilitou bastante a compreensão desta *framework*. A separação entre template, script e estilo em ficheiros .vue ajudou-nos a compreender a estrutura das páginas que fomos desenvolvendo, uma vez que os ficheiros HTML, CSS e Javascript eram todos localizados no mesmo ficheiro.

Por fim, terminamos o projeto com confiança, uma vez que o front-end era uma das poucas áreas que ainda não dominávamos desde o início do curso. Agora, sentimo-nos mais preparados para o mercado de trabalho que nos espera, e acreditamos que será muito mais fácil aprender novas frameworks semelhantes ao Vue.js e aplicá-las num contexto profissional.